

Day 1 — Array Thinking Report

Assessment of problem-solving approach from the First Unique Element question

1. Problem Attempted

Find the first element in an array that appears exactly once. Return -1 if no such element exists.

2. Your Thinking

Your first solution used `arr.count(i) == 1`. This correctly identified the core requirement: a value is valid when its frequency is exactly one, while scanning left-to-right preserves the 'first' requirement.

You then tried to improve the approach by checking only the portion of the array to the right of the current index. That shows a useful optimization instinct: you were trying to avoid repeatedly counting the entire array.

Your second code:

```
def first_unique(arr):
    n = len(arr)
    for i in range(n):
        if arr[i] not in arr[i+1:n]:
            return arr[i]
    return -1
```

3. What You Are Doing Well

Skill	Assessment
Left-to-right traversal	Good — preserves the first qualifying element.
Identifying uniqueness	Good — you understand that another occurrence invalidates a candidate.
Return/break reasoning	Good — you understand that once a valid first candidate is proven, the function can return.
Optimization instinct	Good foundation — you moved away from repeatedly calling <code>count()</code> .

4. Main Reasoning Gap

Your statement — 'before we don't need to check back if it occurs first then only break the function' — contains an important idea, but it needs one condition: a right-side check is sufficient only when you already know the current occurrence is the first occurrence of that value.

Counterexample: `[4, 4, 5]`. At the second 4, there is no 4 to the right, but 4 has already occurred to the left. A right-only test would incorrectly return 4.

So the issue is not your left-to-right idea. The issue is proving that the current occurrence has not already appeared.

5. Current Array Problem-Solving Profile

Area	Level
Basic loops / iteration	Good
First-occurrence reasoning	Good
Frequency reasoning	Good foundation
Edge-case testing	Developing
Time-complexity awareness	Developing
Data-structure optimization	Next focus

6. Recommended Thinking Process

1. Write the simplest correct solution.
2. Ask what operation is repeated unnecessarily.
3. Identify what information can be stored once, such as frequencies.
4. Test the new idea against edge cases: duplicates at the beginning, middle, end, non-adjacent duplicates, and all duplicates.
5. Only then optimize the time complexity.

7. Overall Assessment

Good foundation; the main improvement area is edge-case validation. You are not struggling with basic array traversal. Your next step is to prove that a condition is sufficient before returning from a loop. This is especially important in placement coding because hidden test cases target assumptions like this.

8. Next Skill to Practice

Focus on frequency dictionaries and one-pass array problems. The goal is to keep your current left-to-right reasoning while learning how to reduce repeated scanning from $O(n^2)$ toward $O(n)$.